Contents

Chapter 1

Dynamic Programming

1.1 Longest Common Subsequence (LCS)

Algorithm 1: Longest Common Subsequence

Function LCS(X, Y)**:**

```
    m ← X.length;
    n ← Y.length;
    for i ← 0 to m do
        for j ← 0 to n do
            if i = 0 or j = 0 then
                dp[i][j] ← 0;
            else if X[i] = Y[j] then
                dp[i][j] ← 1 + dp[i − 1][j − 1];
            else
                dp[i][j] ← max(dp[i − 1][j], dp[i][j − j]);
            end
            Print dp[i][w] followed by space;
        end
    end
return dp[n][W];
```

1.2 0/1 Knapsack

Algorithm 2: 0/1 Knapsack

Function Knapsack(*weights*, *values*, n, W)**:**

```
for i ← 0 to n do
    for w ← 0 to c do
        if i = 0 or w = 0 then
            dp[i][w] ← 0;
        else if weights[i − 1] ≤ w then
            dp[i][w] ← max(dp[i − 1][w], dp[i − 1][w − weights[i − 1]] + values[i − 1]);
        else
            dp[i][w] ← dp[i − 1][w];
        end
        Print dp[i][w] followed by space;
    end
end
```

return $dp[n][W]$;

1.3 Coin Change-I

Algorithm 3: Coin Change - (Minimum Coin Required)

Function `MinCoins`(*coins*, n, V)**:**

```
for i ← 0 to n do
    for v ← 0 to V do
        if v = 0 or then
            dp[i][v] ← 0;
        else if i = 0 then
            dp[i][v] ← ∞;
        else if coins[i − 1] ≤ w then
            dp[i][v] = min(dp[i − 1][v],  1 + dp[i][v − coins[i − 1]]);
        else
            dp[i][v] ← dp[i − 1][v];
        end
        Print dp[i][v] followed by space;
    end
end
```

return $dp[n][V]$;

1.4 Coin Change-II

Algorithm 4: Coin Change - (Number of ways)

Function `MinCoins`(*coins*, n, V)**:**

```
for i ← 0 to n do
    for v ← 0 to V do
        if v = 0 or then
            dp[i][v] ← 0;
        else if i = 0 then
            dp[i][v] ← ∞;
        else if coins[i − 1] ≤ w then
            dp[i][v] = min(dp[i − 1][v],  1 + dp[i][v − coins[i − 1]]);
        else
            dp[i][v] ← dp[i − 1][v];
        end
        Print dp[i][v] followed by space;
    end
end
```

return $dp[n][V]$;